Polymove - Microservices Lab 3: API Gateway & Aggregation

Introduction

In the previous labs, you built independent services communicating via REST and gRPC:

- **Polytech** manages students and internship registrations
- **Erasmumu** manages internship offers
- **MI8** collects news and computes city scores

Currently, a client application would need to call multiple services to gather all the information about an offer. This is inefficient and exposes internal architecture details.

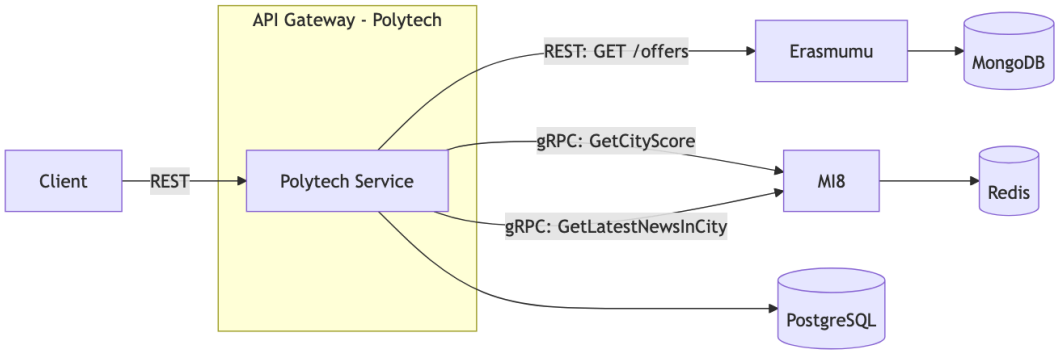
In this lab, you will implement the **API Gateway** and **API Aggregation** patterns to provide a unified, enriched API.

Learning Objectives

By the end of this lab, you will have:

- Implemented an API Gateway in the Polytech service
- Implemented API Aggregation combining data from multiple services
- Understood the benefits and trade-offs of these patterns

Architecture Overview



Part 1: Implement the Aggregated Offers Endpoint

New Endpoint: `GET /offer`

Query Parameters

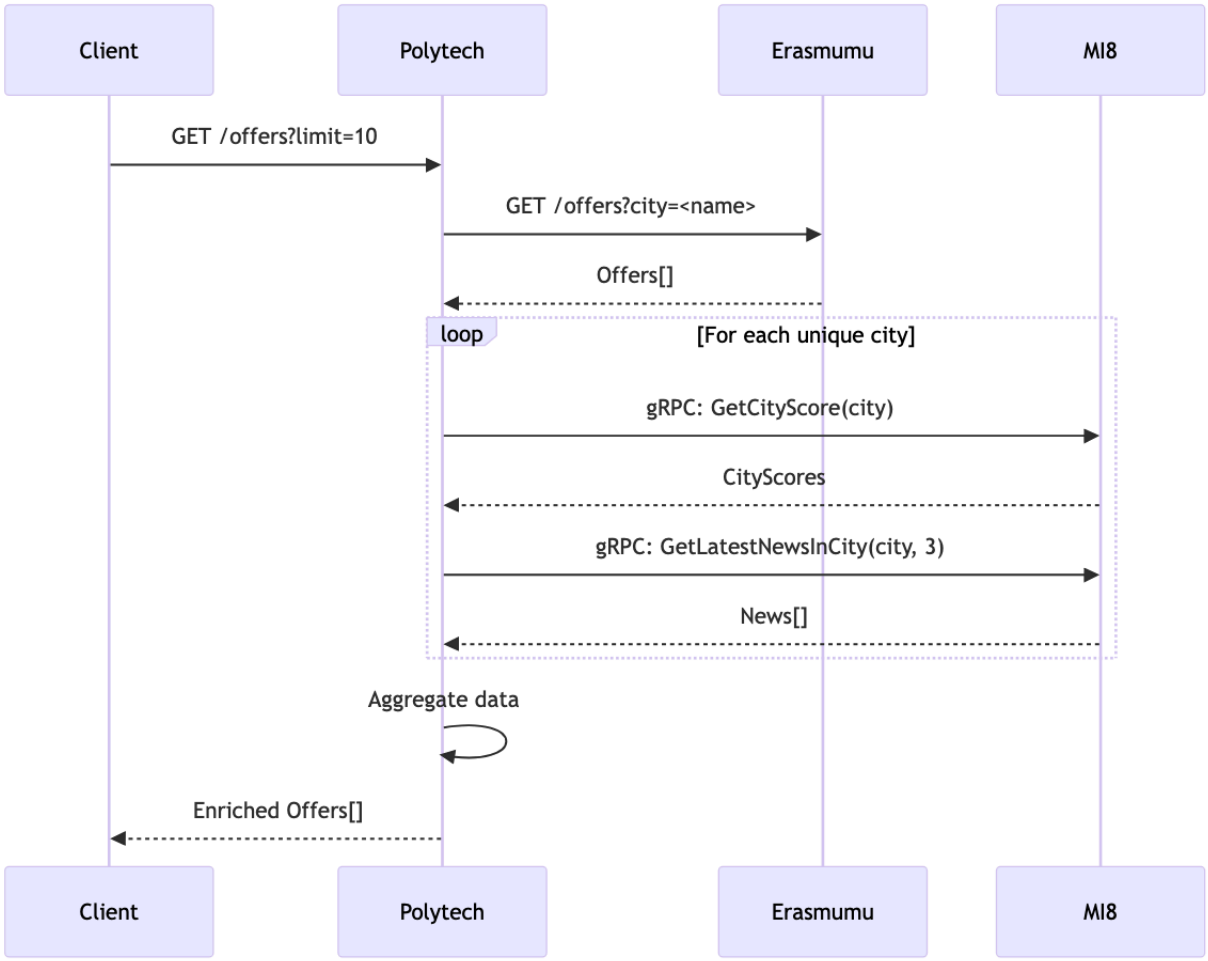
Parameter	Type	Required	Description
`limit`	int	No	Maximum number of offers to return (default: 10)
`city`	string	No	Filter offers by city
`domain`	string	No	Filter offers by domain

Expected Response Format

GET /offer aggregated with cities scores and news

```
{
  "offers": [    // Offers from Erasmumu service
    {
      "id": "offer-42f8c0a9-1e2b-4d3c-a5b6-7e8f90a1b2c3",
      "title": "Senior Backend Developer",
      "link": "https://example.com/jobs/senior-backend-dev-dallas",
      "...other fields": "",
      "city": "dallas",
      "scores": {           // Scores from MI8 service
        "quality_of_life": 1120,
        "economy": 1400,
        "culture": 1250,
        "safety": 700
      },
      "latest_news": [      // News from MI8 service
        {
          "title": "Paris Gentle Mates remporte le major de CoD"
        }
      ]
    },
    {
      "id": "offer-..."
    }
  ]
}
```

Communication flow



Handle Edge Cases

Consider the following scenarios and implement robust error handling in Polytech

- Erasmumu is unavailable
- MI8 has no data for a city
- One service is slow

Part 2: Optimize with Parallel Calls

The Problem

If you have 10 offers across 5 different cities, making sequential calls to MI8 is slow:

- 5 × `GetCityScore` calls
- 5 × `GetLatestNewsInCity` calls
- Total: 10 sequential network calls

Objective

Optimize your aggregation by making parallel calls to MI8.

Implementation

Refactor your code to:

1. Collect all unique cities from the offers
2. Make all MI8 calls **in parallel** (not sequentially)
3. Wait for all responses
4. Assemble the final response

Part 3: Add Student Context Endpoint

Objective

Create an endpoint that provides personalized offer recommendations for a student, combining:

- Student's profile (from Polytech DB)
- Matching offers (from Erasmumu)
- City intelligence (from MI8)

New Endpoint

Method	Endpoint	Description
`GET`	`/students/:id/recommended-offers`	Get personalized offers for a student

Query Parameters

Parameter	Type	Required	Description

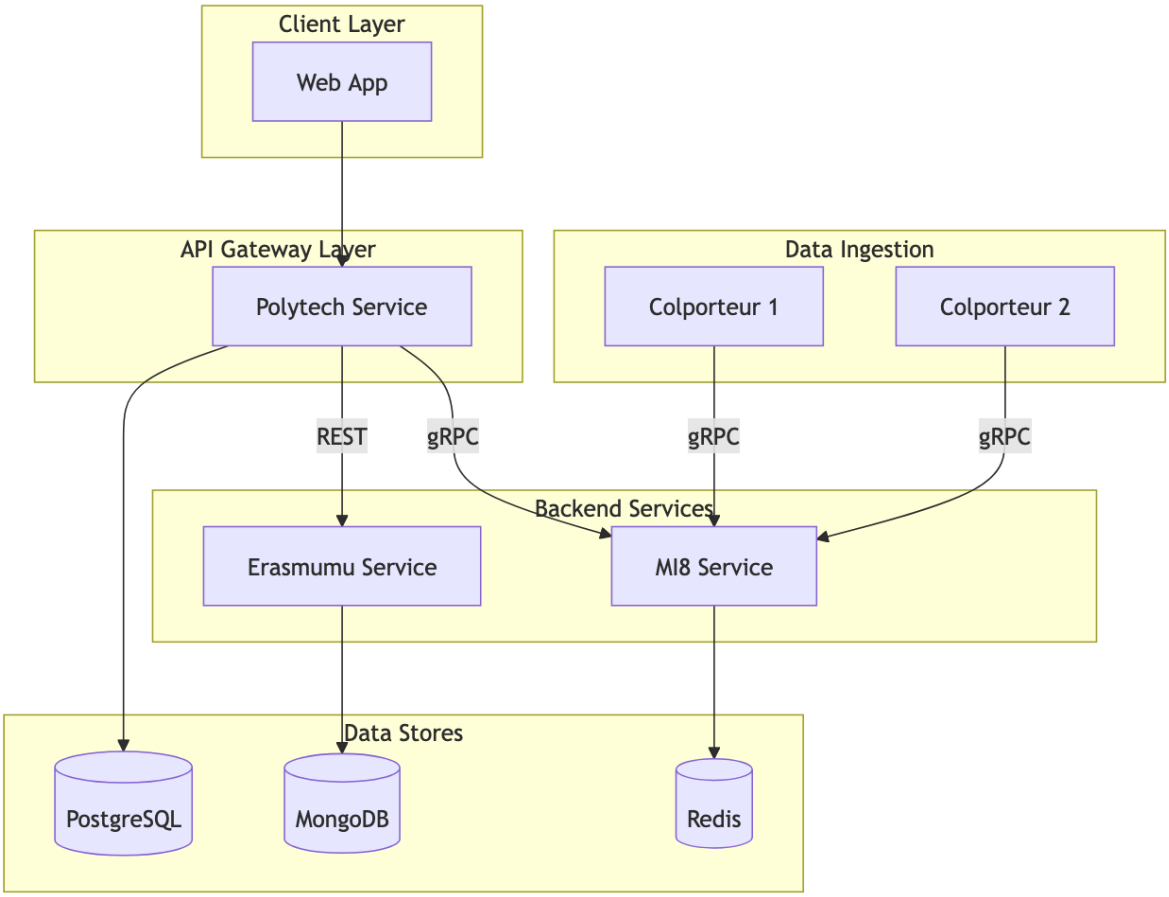
`limit`	int	No	Maximum number of offers (default: 5)
`sort_by`	string	No	Score category to sort by: `safety`, `economy`, `quality_of_life`, `culture`

Example and response format

Student in the Database

Response:


{
"offers": [
{
"id": "offer-xxx",
"title": "DevOps Engineer Internship",
"city": "berlin",
"scores": {
"safety": 1100,
"economy": 1350,
"quality_of_life": 1200,
"culture": 1400
},
"latest_news": [...]
}
]
}



Business Logic

1. Retrieve the student from Polytech's database
2. Fetch offers matching the student's domain from Erasmumu
3. Enrich offers with MI8 data (scores + news)
4. If `sort_by` is provided, sort offers by that score (descending)
5. Return the top N offers

API Summary

Polytech Service (API Gateway)

Method	Endpoint	Description	Source

`POST`	`/student`	Register a student	Local
`GET`	`/student/:id`	Get student by ID	Local
`GET`	`/student?domain=`	Get students by domain	Local
`POST`	`/internship`	Register for internship	Local + Erasmumu
`GET`	`/internship/:id`	Get registration status	Local
`GET`	`/offers`	Get enriched offers	Erasmumu + MI8
`GET`	`/students/:id/recommended-offers`	Get personalized offers	Local + Erasmumu + MI8

Part 4: Implement a Frontend Application

Objective

Build a frontend application that consumes the Polytech API Gateway and displays the aggregated data to users.

Requirements

You are free to use any frontend technology:

- **React** / Next.js
- **Vue.js** / Nuxt
- **Svelte** / SvelteKit

Application Features

Feature 1: Offers Explorer

Create a page that displays available internship offers with their city intelligence.

Requirements:

- Fetch and display offers from `GET /offers`
- Show offer details: title, company, city, salary, dates
- Display the 4 city scores visually (progress bars, gauges, or charts)
- Show the latest news for each city
- Implement filtering by city or domain

Feature 2: Student Dashboard

Create a personalized dashboard for students.

Requirements:

- Student login (simple: just enter student ID)
- Display student profile information
- Fetch and display recommended offers from `GET /students/:id/recommended-offers`
- Allow sorting by score category (safety, economy, quality_of_life, culture)

Feature 3: Internship Application

Allow students to apply for internships.

Requirements:

- "Apply" button on each offer
- Call `POST /internship` with studentId and offerId
- Display the registration result (approved/rejected with message)

Deliverable

Once completed, commit your work to a branch named:
`lab3-api-gateway-polytech`

Be sure to put your repository in public or invite me with my GridexX